



School of Information Technology and Engineering
Security of Operating System

SIS 2

File System and Users Configuration
“Secure SWIFT Logical Terminal Infrastructure”

Checked by: Valyayev Denis

Done by: Faramarz Pedram

Almaty, 2026

Target

Prepare file system structure of project machines, define roles table, manage users and service accounts according to the Secure SWIFT Logical Terminal Infrastructure defined in SIS 1.

1. Define Places of Storing Data

Project machines:

- swift-terminal
- swift-support

1.1 Where do users store their data?

In Fedora Linux:

- Ordinary users store personal data in:

/home/<username>/

Example:

/home/operator1/

/home/admin1/

Permissions:

- Owned by user
- 700 or 750
- No access for other users

1.2 Where do software components store data?

- According to Linux FHS (Filesystem Hierarchy Standard) and official documentation:

PostgreSQL (swift-terminal)

Official Fedora/PostgreSQL paths:

- Data directory:

`/var/lib/pgsql/data/`

- Configuration files:

`/var/lib/pgsql/data/postgresql.conf`

`/var/lib/pgsql/data/pg_hba.conf`

- Service user:

`postgres`

PostgreSQL stores:

- Metadata only (filename, hash, timestamps)
- No confidential files

Nginx (swift-terminal)

Paths:

- Config:

`/etc/nginx/nginx.conf`

`/etc/nginx/conf.d/`

- Logs:

`/var/log/nginx/`

- Web root:

`/usr/share/nginx/html/`

Service user:

`nginx`

Backend API (FastAPI)

Custom systemd service.

Planned paths:

- Application code:

/opt/swift-backend/

- Service file:

/etc/systemd/system/swift-backend.service

- Runs as user:

swiftapi

SWIFT Logical Terminal Storage

Most sensitive data.

Custom secure path:

/swift-lt/

Inside:

/swift-lt/LT1

/swift-lt/LT2

/swift-lt/LT3

/swift-lt/LT4

/swift-lt/LT5

Permissions:

- Owner: swiftapi
- Group: swift-ops
- Mode: 750
- No direct user access

Disk encryption: LUKS

Logs (Support Server)

On swift-support:

- Central logs:

/var/log/remote/

- Backup storage:

/backup/swift/

Owned by:

backup

2. Prepare Configuration Files

2.1 Configuration Storage Locations

Software	Config Location	Format
PostgreSQL	/var/lib/pgsql/data/	.conf
Nginx	/etc/nginx/	.conf
Backend API	/opt/swift-backend/.env	ENV
Rsyslog	/etc/rsyslog.conf	.conf
Systemd service	/etc/systemd/system/	.service

2.2 Configuration File Formats

- .conf → plain text
- .service → systemd unit format
- .env → KEY=VALUE
- .json (needed for backend)

2.3 Sample Configuration Files

PostgreSQL (postgresql.conf sample)

```
listen_addresses = 'localhost'  
port = 5432
```

```
max_connections = 100
logging_collector = on
log_directory = 'log'
```

Nginx (swift.conf sample)

```
server {
    listen 443 ssl;
    server_name swift.local;

    root /usr/share/nginx/html;

    location /api/ {
        proxy_pass http://127.0.0.1:8000;
    }
}
```

Backend systemd service

[Unit]

Description=SWIFT Backend API

After=network.target

[Service]

User=swiftapi

Group=swiftapi

WorkingDirectory=/opt/swift-backend

ExecStart=/usr/bin/uvicorn main:app --host 127.0.0.1 --port 8000

Restart=always

[Install]

WantedBy=multi-user.target

3. Role Table

3.1 System Roles

Roles include:

- Admin
- Security Officer
- Operator
- Auditor
- Service Accounts

3.2 Role Table

Role	Description	Permissions	Superuser Permissions
admin	System administrator	Install software, manage users, SSH	sudo ALL
security	Manages logs and audit	Read logs, manage rsyslog	sudo limited (journalctl, firewall-cmd)
operator	SWIFT file operator	Upload/download via UI	none
auditor	Read-only log access	Read logs only	none
postgres	DB service account	Own DB files	none
nginx	Web server account	Serve files	none
swiftapi	Backend service	Access LT directories	none
backup	Backup account	Store backups	none

4. Create Groups and Users

MACHINE 1: swift-terminal

This is the main SWIFT server.

You must create:

1. Create Groups (run as root or sudo)

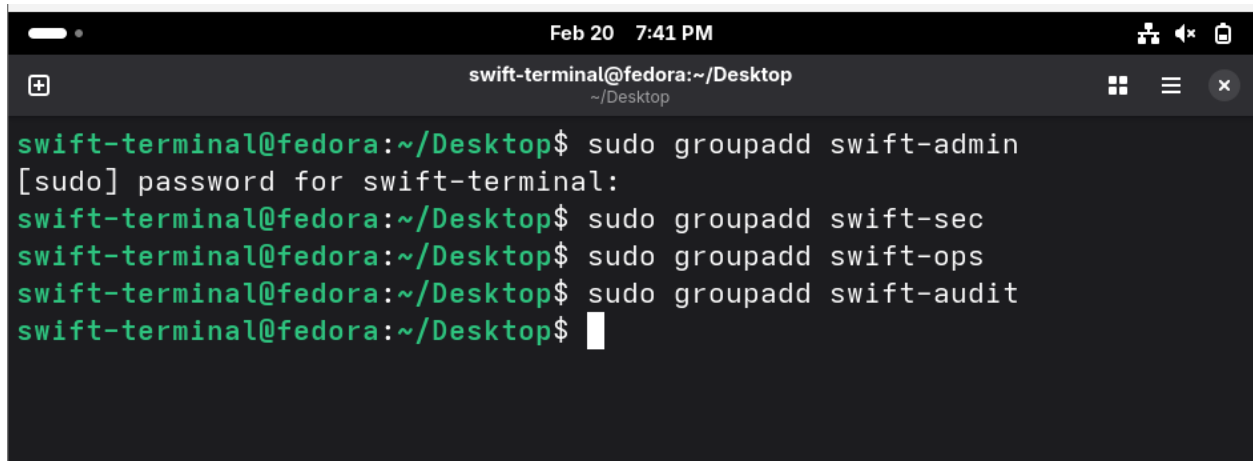
```
sudo groupadd swift-admin
```

```
sudo groupadd swift-sec
```

```
sudo groupadd swift-ops
```

```
sudo groupadd swift-audit
```

These are human role groups.

A terminal window titled 'swift-terminal@fedora:~/Desktop' with a timestamp 'Feb 20 7:41 PM'. The terminal shows the execution of four 'sudo groupadd' commands to create the groups 'swift-admin', 'swift-sec', 'swift-ops', and 'swift-audit'. The first command is followed by a password prompt for 'swift-terminal'.

```
Feb 20 7:41 PM
swift-terminal@fedora:~/Desktop
swift-terminal@fedora:~/Desktop$ sudo groupadd swift-admin
[sudo] password for swift-terminal:
swift-terminal@fedora:~/Desktop$ sudo groupadd swift-sec
swift-terminal@fedora:~/Desktop$ sudo groupadd swift-ops
swift-terminal@fedora:~/Desktop$ sudo groupadd swift-audit
swift-terminal@fedora:~/Desktop$
```

2. Create Human Users

Admin

```
sudo useradd -m -G swift-admin admin1
```

```
sudo passwd admin1
```



```
Feb 20 7:43 PM
swift-terminal@fedora:~/Desktop
swift-terminal@fedora:~/Desktop$ sudo useradd -m -G swift-admin admin1
swift-terminal@fedora:~/Desktop$ sudo passwd admin1
sudo: passwd: command not found
swift-terminal@fedora:~/Desktop$ sudo passwd admin1
New password:
Retype new password:
passwd: password updated successfully
swift-terminal@fedora:~/Desktop$
```

Security Officer

```
sudo useradd -m -G swift-sec sec1
```

```
sudo passwd sec1
```

```
Feb 20 7:44 PM
swift-terminal@fedora:~/Desktop
-s, --stdin          read new token from stdin
swift-terminal@fedora:~/Desktop$ sudo useradd -m -G swift-sec sec1
useradd: user 'sec1' already exists
swift-terminal@fedora:~/Desktop$ sudo passwd sec1
New password:
Retype new password:
passwd: password updated successfully
swift-terminal@fedora:~/Desktop$
swift-terminal@fedora:~/Desktop$
```

Operator

```
sudo useradd -m -G swift-ops operator1
```

```
sudo passwd operator1
```

```
Feb 20 7:58 PM
swift-terminal@fedora:~/Desktop
useradd: group swift-ops does not exist
swift-terminal@fedora:~/Desktop$ sudo useradd -m -G swift-ops operator1
swift-terminal@fedora:~/Desktop$ sudo passwd operator1
New password:
Retype new password:
passwd: password updated successfully
swift-terminal@fedora:~/Desktop$
swift-terminal@fedora:~/Desktop$
swift-terminal@fedora:~/Desktop$
swift-terminal@fedora:~/Desktop$
```

Auditor

```
sudo useradd -m -G swift-audit audit1
```

```
sudo passwd audit1
```

```
Feb 20 7:59 PM
swift-terminal@fedora:~/Desktop
swift-terminal@fedora:~/Desktop$ sudo useradd -m -G swift-audit audit1
[sudo] password for swift-terminal:
Sorry, try again.
[sudo] password for swift-terminal:
swift-terminal@fedora:~/Desktop$ sudo passwd audit1
New password:
Retype new password:
passwd: password updated successfully
swift-terminal@fedora:~/Desktop$
```

3. Assign Sudo Rights (IMPORTANT)

Open:

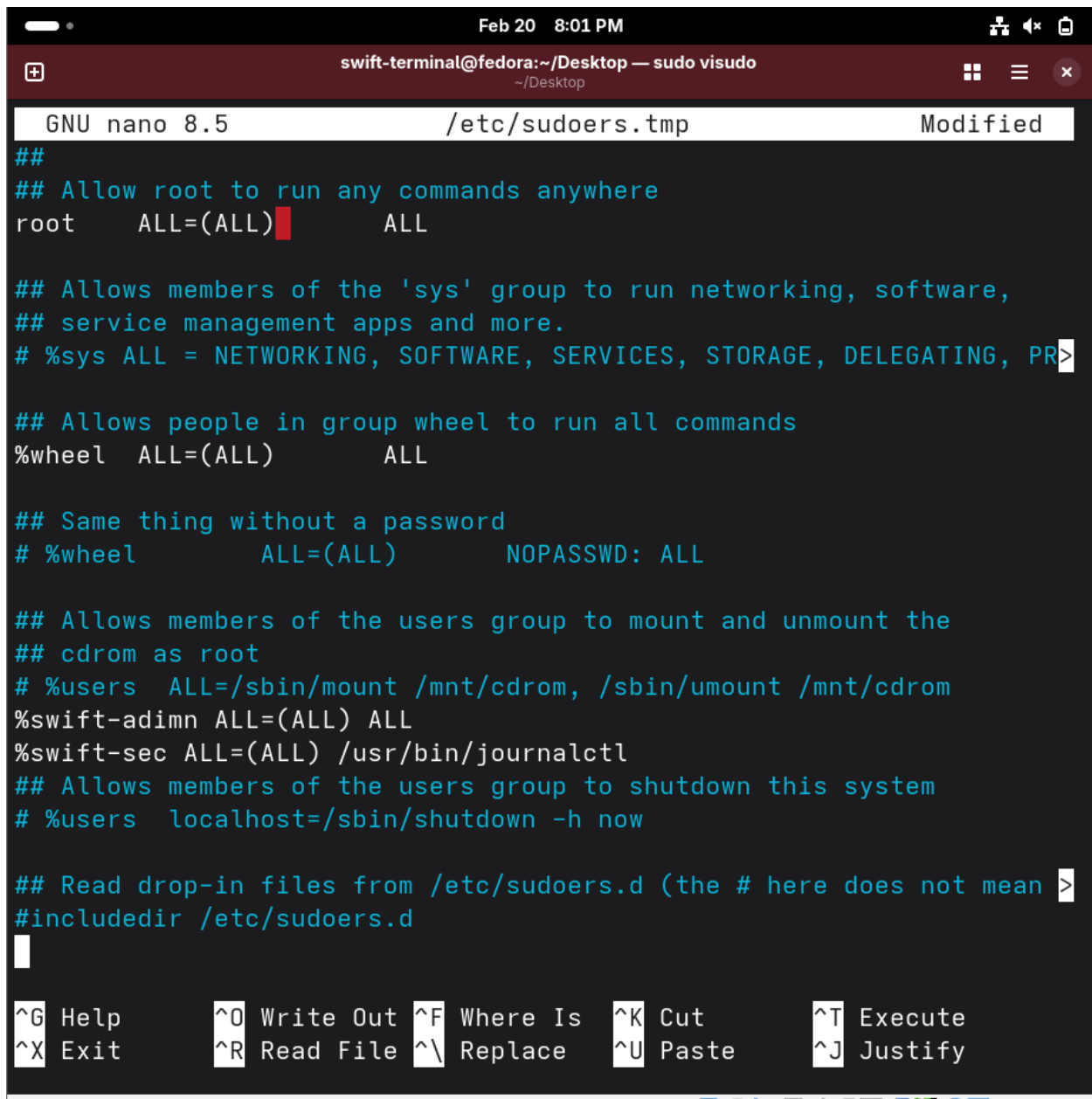
```
sudo visudo
```

Add:

```
%swift-admin ALL=(ALL) ALL
```

Optional limited sudo for security:

```
%swift-sec ALL=(ALL) /usr/bin/journalctl
```



```
GNU nano 8.5 /etc/sudoers.tmp Modified
##
## Allow root to run any commands anywhere
root    ALL=(ALL)      ALL

## Allows members of the 'sys' group to run networking, software,
## service management apps and more.
# %sys ALL = NETWORKING, SOFTWARE, SERVICES, STORAGE, DELEGATING, PR>

## Allows people in group wheel to run all commands
%wheel  ALL=(ALL)      ALL

## Same thing without a password
# %wheel          ALL=(ALL)          NOPASSWD: ALL

## Allows members of the users group to mount and unmount the
## cdrom as root
# %users  ALL=/sbin/mount /mnt/cdrom, /sbin/umount /mnt/cdrom
%swift-adimn ALL=(ALL) ALL
%swift-sec ALL=(ALL) /usr/bin/journalctl
## Allows members of the users group to shutdown this system
# %users  localhost=/sbin/shutdown -h now

## Read drop-in files from /etc/sudoers.d (the # here does not mean >
#include:: /etc/sudoers.d

```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

4. Create Service Accounts (swift-terminal)

These are system users (no login).

Backend user

```
sudo useradd -r -s /sbin/nologin swiftapi
```

```
swift-terminal@fedora:~/Desktop$ sudo useradd -r -s /sbin/nologin swiftapi
```

PostgreSQL (usually auto-created during install)

Check:

```
id postgres
```

```
swift-terminal@fedora:~/Desktop$ sudo dnf install postgresql-server
Updating and loading repositories:
Repositories loaded.
Package Arch Version Reposit Size
Installing:
postgresql-server x86_64 18.1-1.fc43 updates 31.7 MiB
Installing dependencies:
postgresql x86_64 18.1-1.fc43 updates 8.3 MiB
postgresql-private-libs x86_64 18.1-1.fc43 updates 372.0 KiB

Transaction Summary:
Installing: 3 packages

Total size of inbound packages is 10 MiB. Need to download 10 MiB.
After this operation, 40 MiB extra will be used (install 40 MiB, remove 0 B).
Is this ok [y/N]: y
[1/3] postgresql-private-lib 100% | 33.4 KiB/s | 159.2 KiB | 00m05s
[2/3] postgresql-0:18.1-1.fc 100% | 304.9 KiB/s | 2.0 MiB | 00m07s
[3/3] postgresql-server-0:18 4% | 108.4 KiB/s | 331.7 KiB | 01m07s
-----
[2/3] Total 25% | 1.4 MiB/s | 2.5 MiB | 00m04s
```

```
swift-terminal@fedora:~/Desktop$ id postgres
uid=26(postgres) gid=26(postgres) groups=26(postgres)
```

Nginx (auto-created)

```
id nginx
```

```
swift-terminal@fedora:~/Desktop$ sudo dnf install postgresql-server n
ginx
Updating and loading repositories:
Repositories loaded.
Package "postgresql-server-18.1-1.fc43.x86_64" is already installed.
```

Package	Arch	Version	Reposi	Size
Upgrading:				
pcre2	x86_64	10.47-1.fc43	update	702.6 KiB
replacing pcre2	x86_64	10.46-1.fc43	0e44b8	697.7 KiB
pcre2-syntax	noarch	10.47-1.fc43	update	281.9 KiB
replacing pcre2-syntax	noarch	10.46-1.fc43	0e44b8	275.3 KiB
pcre2-utf16	x86_64	10.47-1.fc43	update	639.2 KiB
replacing pcre2-utf16	x86_64	10.46-1.fc43	0e44b8	626.3 KiB
pcre2-utf32	x86_64	10.47-1.fc43	update	611.1 KiB
replacing pcre2-utf32	x86_64	10.46-1.fc43	0e44b8	602.2 KiB
Installing:				
nginx	x86_64	2:1.28.2-1.fc43	update	119.4 KiB
Installing dependencies:				
nginx-core	x86_64	2:1.28.2-1.fc43	update	1.8 MiB
nginx-filesystem	noarch	2:1.28.2-1.fc43	update	141.0 B
nginx-mimetypes	noarch	2.1.54-9.fc43	fedora	46.6 KiB
Transaction Summary:				
Installing:	4 packages			
Upgrading:	4 packages			
Replacing:	4 packages			

```
swift-terminal@fedora:~/Desktop$ id nginx
uid=968/nginx gid=968/nginx groups=968/nginx)
swift-terminal@fedora:~/Desktop$
```

5. Create SWIFT Storage Directory

```
sudo mkdir /swift-lt
sudo mkdir /swift-lt/LT1
sudo mkdir /swift-lt/LT2
sudo mkdir /swift-lt/LT3
sudo mkdir /swift-lt/LT4
sudo mkdir /swift-lt/LT5
```

```

swift-terminal@fedora:~/Desktop$ sudo mkdir /swift-lt
mkdir: cannot create directory '/swift-lt': File exists
swift-terminal@fedora:~/Desktop$ sudo mkdir /swift-lt/LT1
swift-terminal@fedora:~/Desktop$ sudo mkdir /swift-lt/LT2
swift-terminal@fedora:~/Desktop$ sudo mkdir /swift-lt/LT3
swift-terminal@fedora:~/Desktop$ sudo mkdir /swift-lt/LT4
swift-terminal@fedora:~/Desktop$ sudo mkdir /swift-lt/LT5
swift-terminal@fedora:~/Desktop$

```

Assign ownership:

```
sudo chown -R swiftapi:swift-ops /swift-lt
```

```
sudo chmod -R 750 /swift-lt
```

Meaning:

- swiftapi → full access
- swift-ops → read/execute
- others → no access

```

swift-terminal@fedora:~/Desktop$ ls -ld /swift-lt
drwxr-xr-x. 1 root root 30 Feb 20 20:10 /swift-lt
swift-terminal@fedora:~/Desktop$ sudo chown -R swiftapi:swift-ops /swift-lt
swift-terminal@fedora:~/Desktop$ sudo chmod -R 750 /swift-lt
swift-terminal@fedora:~/Desktop$ ls -ld /swift-lt
drwxr-x---. 1 swiftapi swift-ops 30 Feb 20 20:10 /swift-lt
swift-terminal@fedora:~/Desktop$

```

MACHINE 2: swift-support

This machine handles:

- Logs
- Backups
- Admin jump host

1. Create Groups

```
sudo groupadd swift-admin
sudo groupadd swift-sec
sudo groupadd swift-audit
```

(No need for swift-ops here — operators do not log into support server)

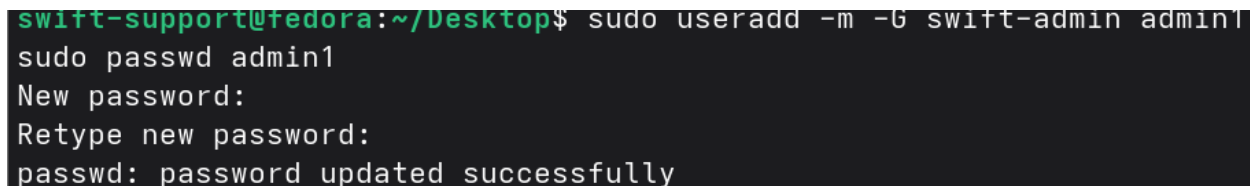
A terminal window titled 'swift-support@fedora:~/Desktop' with a dark background and green text. The window shows the execution of three 'sudo groupadd' commands to create the groups 'swift-admin', 'swift-sec', and 'swift-audit'. The prompt is 'swift-support@fedora:~/Desktop\$' and the output for the first command is '[sudo] password for swift-support:'.

```
Feb 20 8:22 PM
swift-support@fedora:~/Desktop
swift-support@fedora:~/Desktop$ sudo groupadd swift-admin
[sudo] password for swift-support:
swift-support@fedora:~/Desktop$ sudo groupadd swift-sec
swift-support@fedora:~/Desktop$ sudo groupadd swift-audit
swift-support@fedora:~/Desktop$
```

2. Create Users

Admin:

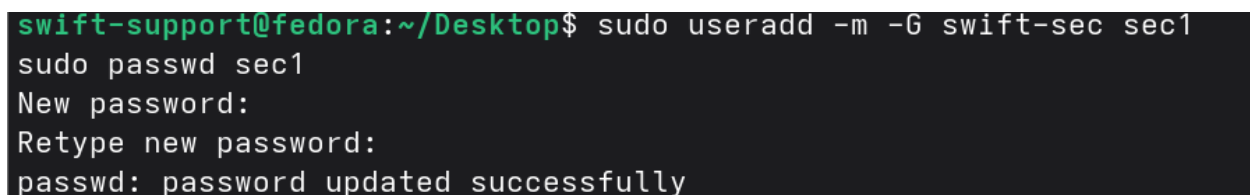
```
sudo useradd -m -G swift-admin admin1
sudo passwd admin1
```

A terminal window showing the creation of the 'admin1' user. The prompt is 'swift-support@fedora:~/Desktop\$'. The commands are 'sudo useradd -m -G swift-admin admin1' and 'sudo passwd admin1'. The output for 'passwd' shows 'New password:', 'Retype new password:', and 'passwd: password updated successfully'.

```
swift-support@fedora:~/Desktop$ sudo useradd -m -G swift-admin admin1
swift-support@fedora:~/Desktop$ sudo passwd admin1
New password:
Retype new password:
passwd: password updated successfully
```

Security:

```
sudo useradd -m -G swift-sec sec1
sudo passwd sec1
```

A terminal window showing the creation of the 'sec1' user. The prompt is 'swift-support@fedora:~/Desktop\$'. The commands are 'sudo useradd -m -G swift-sec sec1' and 'sudo passwd sec1'. The output for 'passwd' shows 'New password:', 'Retype new password:', and 'passwd: password updated successfully'.

```
swift-support@fedora:~/Desktop$ sudo useradd -m -G swift-sec sec1
swift-support@fedora:~/Desktop$ sudo passwd sec1
New password:
Retype new password:
passwd: password updated successfully
```

Auditor:

```
sudo useradd -m -G swift-audit audit1
sudo passwd audit1
```

```
swift-support@fedora:~/Desktop$ sudo useradd -m -G swift-audit audit1
sudo passwd audit1
New password:
Retype new password:
passwd: password updated successfully
swift-support@fedora:~/Desktop$
```

3. Create Backup Service User

```
sudo useradd -r -s /sbin/nologin backup
```

Create backup directory:

```
sudo mkdir -p /backup/swift
sudo chown -R backup:backup /backup
sudo chmod -R 700 /backup
```

```
swift-support@fedora:~/Desktop$ sudo useradd -r -s /sbin/nologin backup
swift-support@fedora:~/Desktop$ sudo mkdir -p /backup/swift
sudo chown -R backup:backup /backup
sudo chmod -R 700 /backup
```

Conclusions

In this Individual Study, I prepared the file system structure and user management configuration for the Secure SWIFT Logical Terminal Infrastructure.

I defined correct storage locations for users and software components according to Linux standards. A secure directory /swift-lt was created on the swift-terminal machine to store sensitive SWIFT files with proper ownership and permissions.

I also defined configuration file locations and formats for the planned software components. A role-based access control model was implemented, including

administrator, security, operator, auditor, and dedicated service accounts. Permissions were assigned according to the principle of least privilege, and superuser rights were limited only to administrators.

All required groups and users were created on the appropriate machines, and access control was configured correctly.

As a result, the system is properly structured, secure, and ready for the next stage of implementation.

Checking command:

- `cat /etc/passwd`
- `cat /etc/group`
- `ls -ld /swift-lt`
- `sudo -l`
- `id swiftapi`
- `ls -ld /backup`